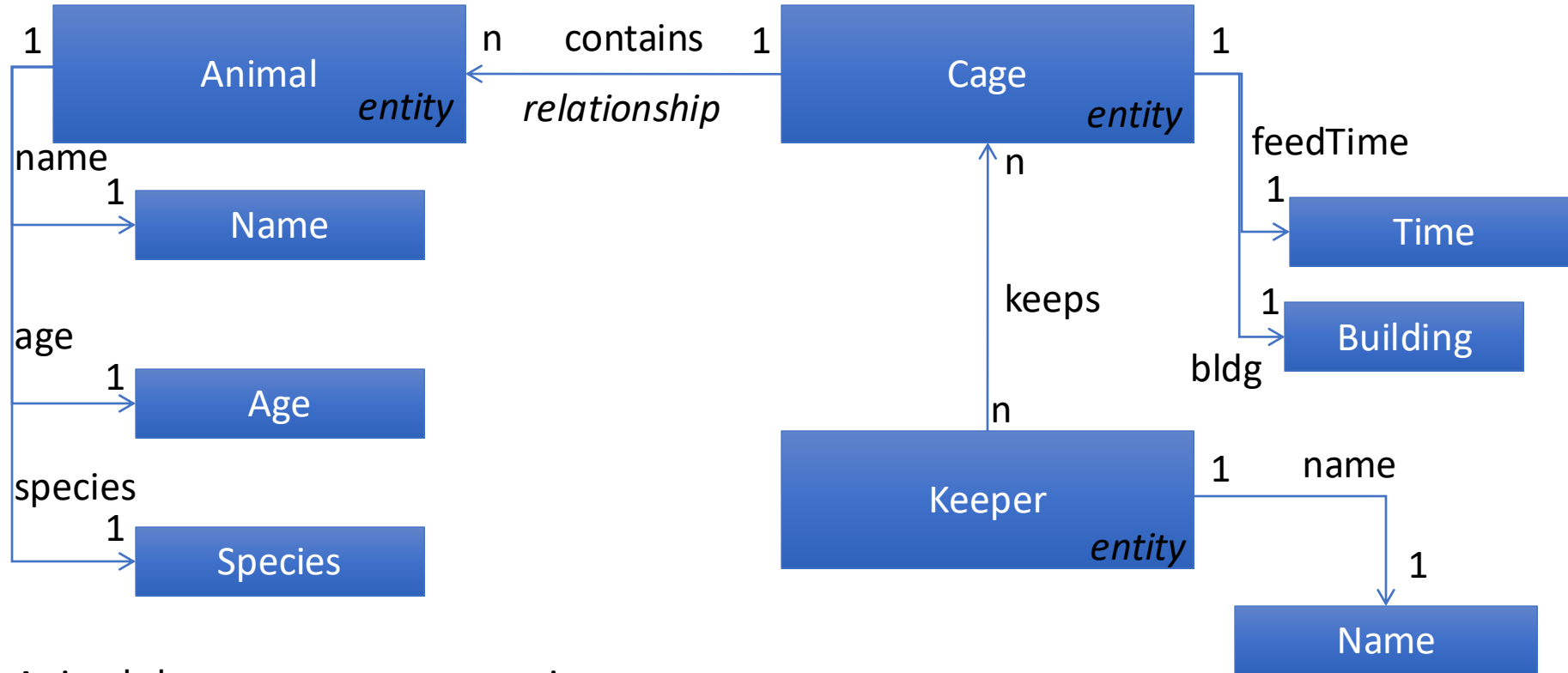


Database Design and Normalization



Back To The Zoo



Animals have names, ages, species

Keepers have names

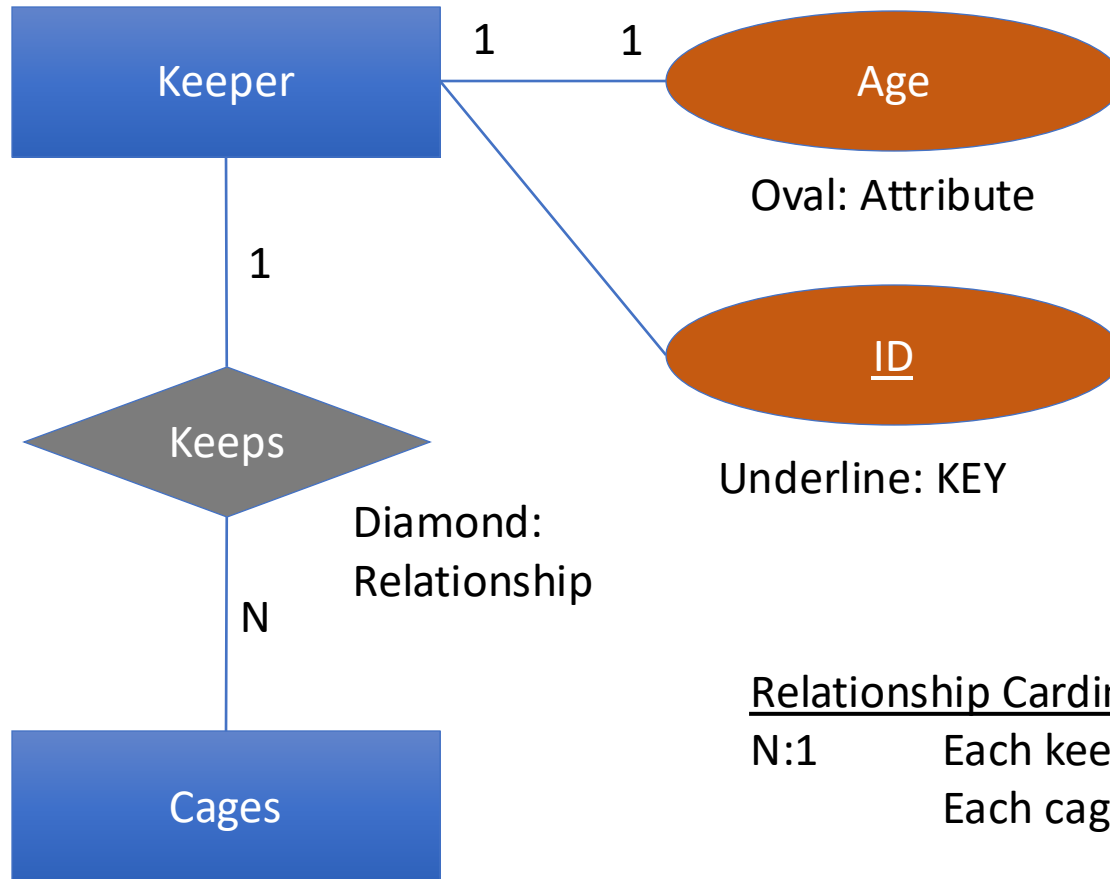
Cages have cleaning times, buildings

Animals are in 1 cage; cages have multiple animals

Keepers keep multiple cages, cages kept by multiple keepers

ER Diagrams

Rectangle: Entity



Oval: Attribute

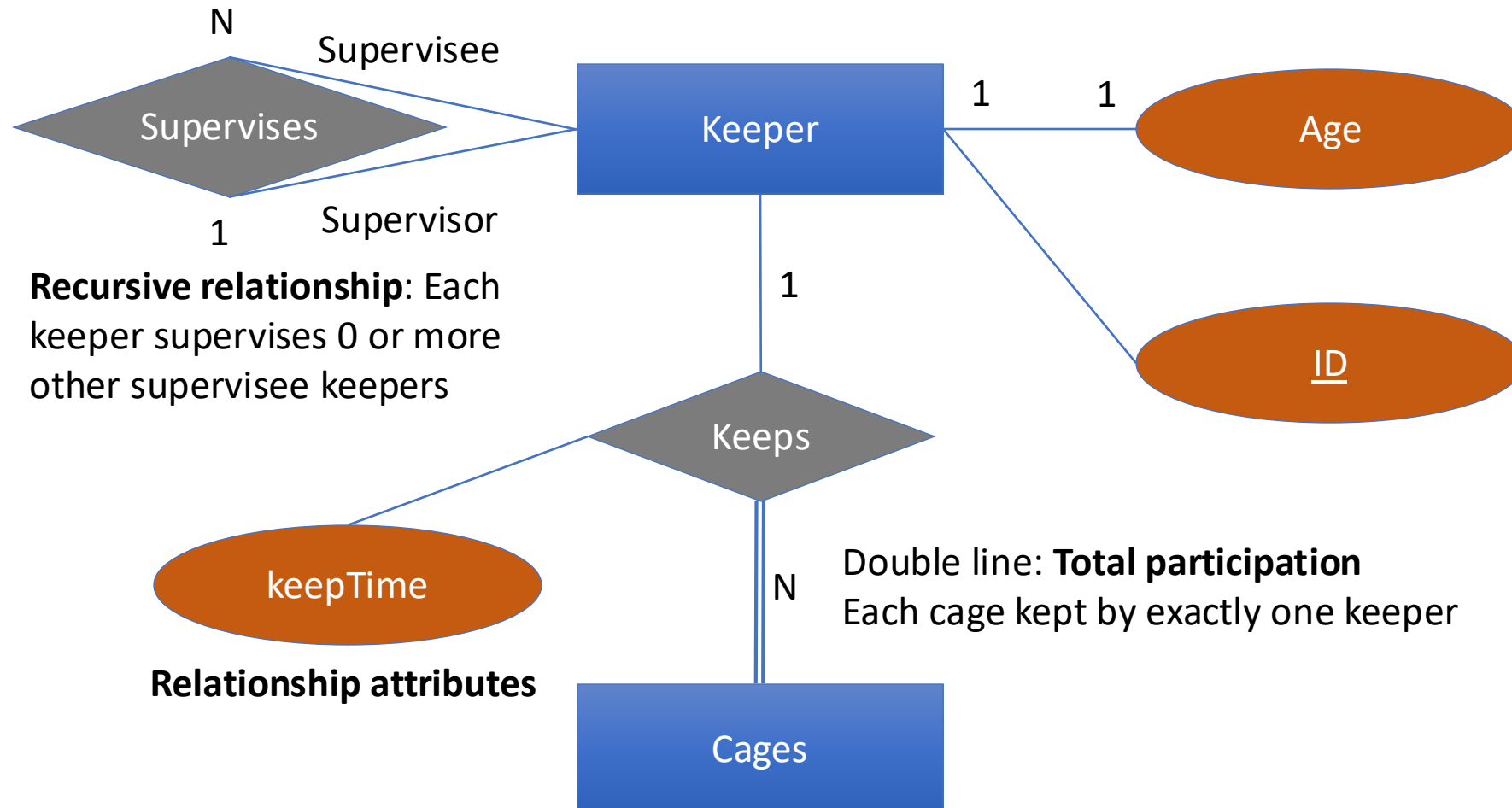
Underline: KEY

Relationship Cardinality

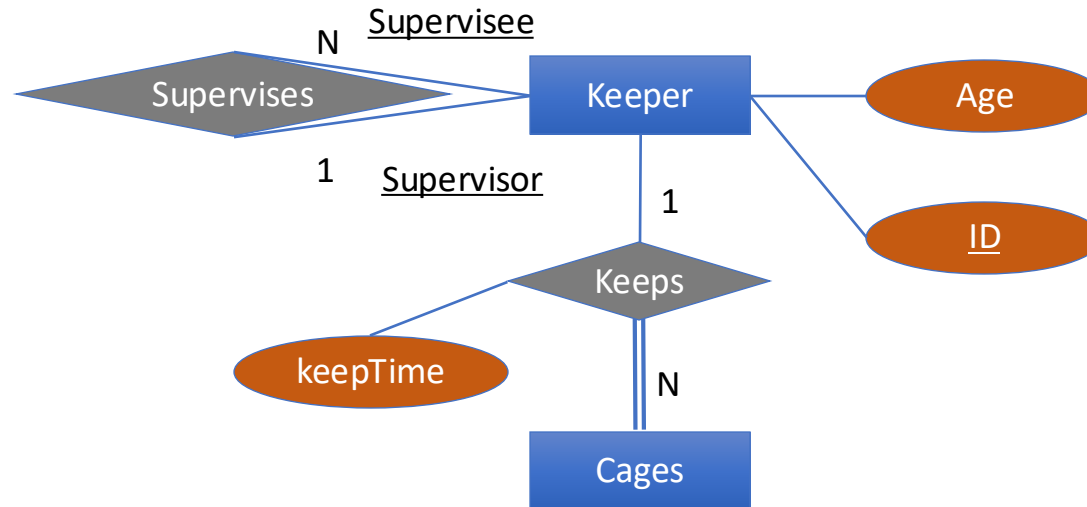
N:1 Each keeper keeps 0 or more cages
 Each cage kept by 0 or 1 keeper

N:N Each keeper keeps 0 or more cages
 Each cage kept by 0 or more keeper

More ER Modeling



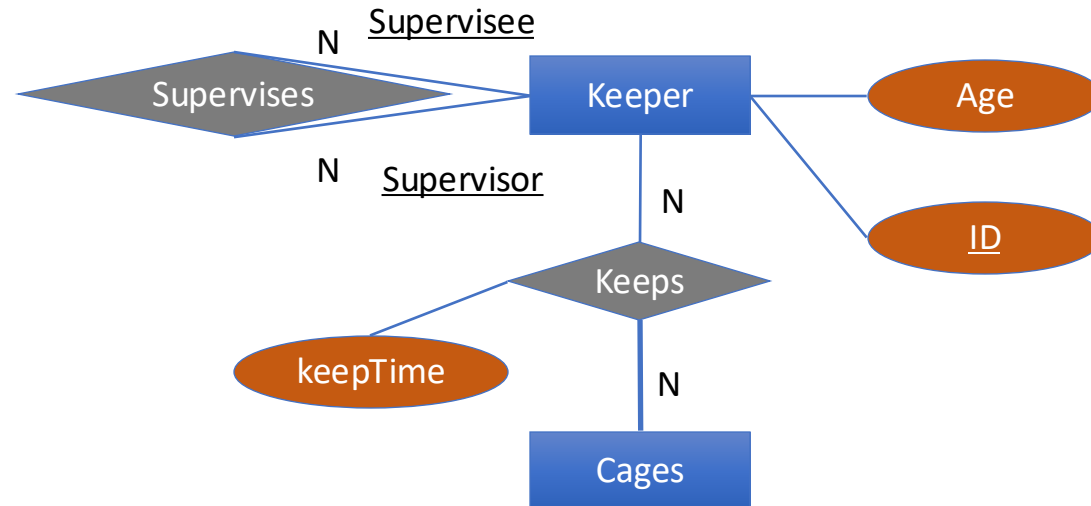
Converting to Relations



Keepers: (ID, age, ... supervisor REFERENCES Keepers.ID ...)

Cages: (keptby NON NULL REFERENCES Keepers.id, keepTime, ...)

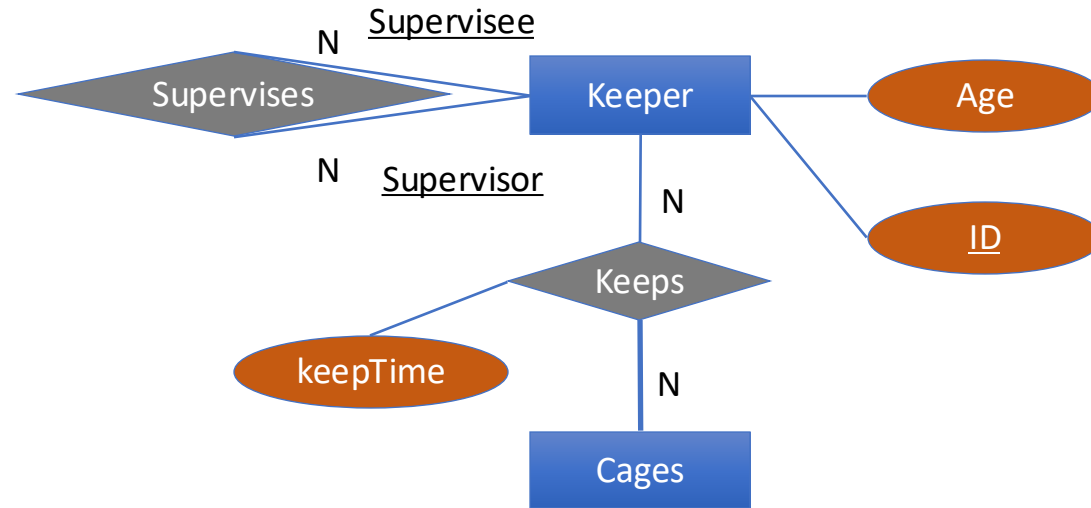
Practice Question



What is the schema design if we change the relationships to be N:N?

(I.e., employees can have multiple supervisors, cages can be kept by multiple keepers?)

Solution



Keeps: (ID, age)

Cages: (CageID, ...)

Keeps: (kid, cid, keepTime)

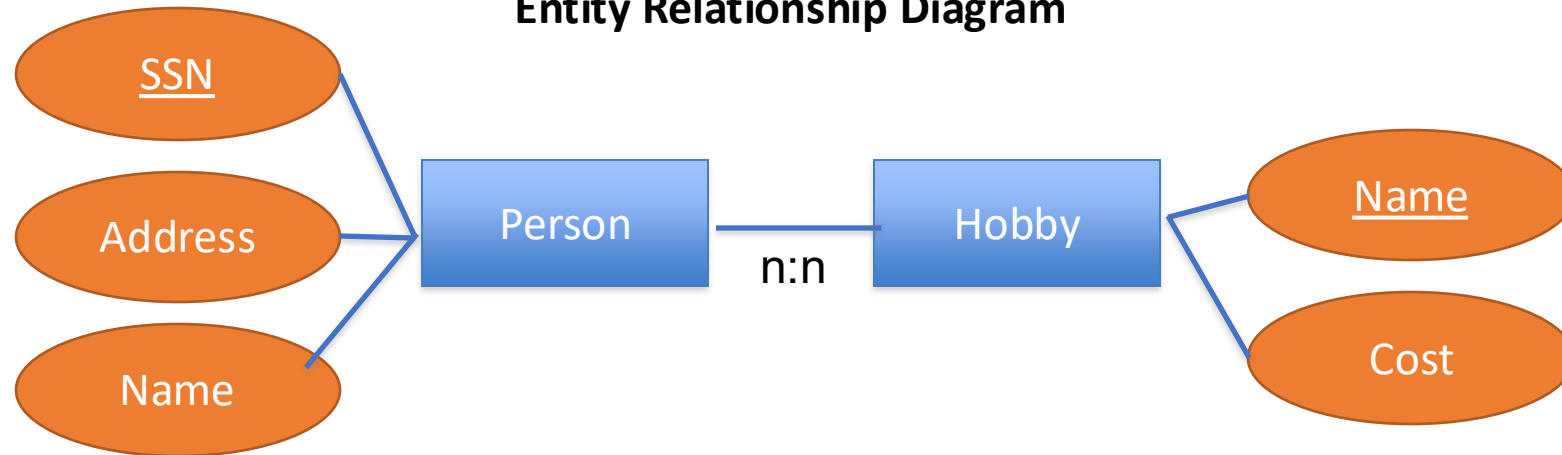
Supervises: (supervisor kid, supervisee kid)

Hobbies Example

- Consider a database about people & their hobbies



Entity Relationship Diagram



People have names and addresses, hobbies have costs

People can have multiple hobbies, and hobbies can be practiced by multiple people

Hobby DB, Attempt 1

SSN	Name	Address	Hobby	Cost
123	john	main st	dolls	\$
123	john	main st	bugs	\$
345	mary	lake st	tennis	\$\$
456	joe	first st	dolls	\$

Table key is
Hobby, SSN

“Wide” schema

- has redundancy (wasted space)
- anomalies in the presence of updates, inserts, and deletes
- + avoids joins

Types of Anomalies

- **Update anomaly**
 - E.g., address needs to change in several places
 - Creates possibility for inconsistency
- **Insertion anomaly** - what if we want to add someone with no hobby?
 - Can we use NULLs?
 - Problem: hobby is a part of the key!
- **Solution: “Normalize”!**

Hobby DB Attempt 2

SSN	Hobby
123	dolls
123	bugs
345	tennis
456	dolls

SSN	Name	Address	Cost
123	john	main st	\$
345	mary	lake st	\$\$
456	joe	first st	\$

- **“Lossy decomposition”**
- No redundancy, but we have lost some information (cost of hobbies)

Normalization

- *Normalized*: a schema that is redundancy free
 - As much as possible
- And that preserves dependencies
 - As much as possible
- Several methods:
 - ER Diagrams
 - Use functional dependencies and normal forms

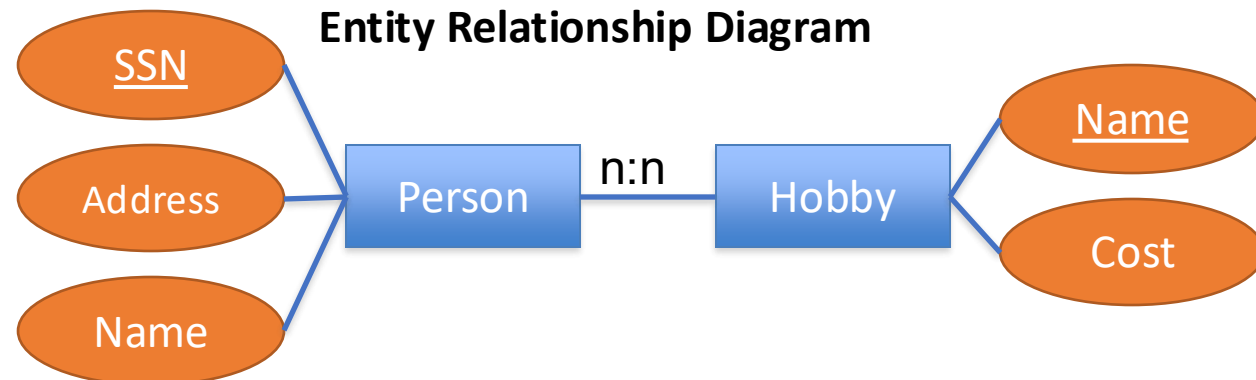
Schema From ER Diagram

SSN	Name	Address	Hobby	Cost	st
123	john	main st	dolls	\$	
123	john	main st	bugs	\$	
345	mary	lake st	tennis	\$\$	
SSN → Name, Address	john	first st	dolls	Hobby → Cost	

Hobby	SSN
dolls	123
bugs	123
tennis	345
dolls	456

Hobby, SSN → Hobby, SSN

Schema is free of anomalies and redundancy



Why Does Redundancy Arise?

- When a subset of attributes are uniquely determined by a *subkey*
 - E.g., SSN determines name, address
 - Key is SSN, Hobby
 - Each row with same SSN will duplicate data!

SSN	Name	Address	Hobby	Cost
123	john	main st	dolls	\$
123	john	main st	bugs	\$
345	mary	lake st	tennis	\$\$
456	joe	first st	dolls	\$

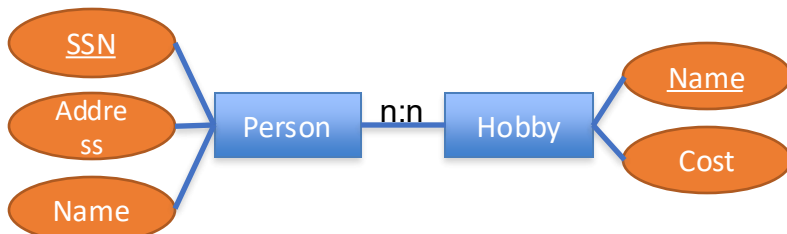
Functional Dependencies

- $X \rightarrow Y$
- Attributes X uniquely determine Y
 - I.e., for every pair of instances x_1, x_2 in X, with y_1, y_2 in Y...
if $x_1=x_2$, then $y_1=y_2$
- For Hobbies, we have:
 1. $SSN \rightarrow \text{Name, Addr}$
 2. $\text{Hobby} \rightarrow \text{Cost}$
 3. $SSN, \text{Hobby} \rightarrow \text{Name, Addr, Cost}$
- 1 & 2 imply 3, by union (“Armstrong’s Axioms”)
 - $F1: A \rightarrow B, F2: C \rightarrow D, F1 \cup F2 = A \cup C \rightarrow B \cup D$

FDs are a Property of the Application, Not the Data

- Can't necessarily tell the FDs by looking at the data
- Given the FDs, can verify that the data satisfies them!
- Example: Is cost a property of hobby or person?

SSN	Name	Address	Hobby	Cost
123	john	main st	dolls	\$
123	john	main st	bugs	\$
345	mary	lake st	tennis	\$\$
456	joe	first st	dolls	\$



ER Diagram forces DB designer to model this!

Question

Consider the following Excel spreadsheet

employee			project		billed			
eid	name	phone	pid	title	date	hours	rate	total
1	Mike	410-XXXX	1	Data modeling	9/9	3	\$150/h	\$450
1	Mike	410-XXXX	1	Data modeling	9/10	2	\$150/h	\$300
1	Mike	410-XXXX	2	Model Testing	9/10	3	\$25/h	\$150
2	Tianyu	510-XXXX	1	Data modeling	9/10	2	\$100/h	\$300
2	Tianyu	510-XXXX	3	Model Training	9/10	4	\$100/h	\$400

Write down all possible functional dependencies

<https://clicker.mit.edu/6.8530/>

Consider the following Excel spreadsheet

employee			project		billed			
eid	name	phone	pid	title	date	hours	rate	total
1	Mike	410-XXXX	1	Data modeling	9/9	3	\$150/h	\$450
1	Mike	410-XXXX	1	Data modeling	9/10	2	\$150/h	\$300
1	Mike	410-XXXX	2	Model Testing	9/10	3	\$25/h	\$150
2	Tianyu	510-XXXX	1	Data modeling	9/10	2	\$100/h	\$300
2	Tianyu	510-XXXX	3	Model Training	9/10	4	\$100/h	\$400

Select all valid functional dependencies.

A: eid → pid, title

B: eid → name, phone

C: pid → title

D: Eid, pid → rate

E: date → pid

F: hours, date → rate

G: eid, pid, date → hours, total

H: hours, rate → total

I: phone → eid, name

<https://clicker.mit.edu/6.8530/>

Consider the following Excel billing spreadsheet

employee			project		billed			
eid	name	phone	pid	title	date	hours	rate	total
1	Mike	410-XXXX	1	Data modeling	9/9	3	\$150/h	\$450
1	Mike	410-XXXX	1	Data modeling	9/10	2	\$150/h	\$300
1	Mike	410-XXXX	2	Model Testing	9/10	3	\$25/h	\$150
2	Tianyu	510-XXXX	1	Data modeling	9/10	2	\$100/h	\$300
2	Tianyu	510-XXXX	3	Model Training	9/10	4	\$100/h	\$400

Solution

eid -> name, phone

pid -> title

Eid,pid->rate

eid,pid,date->hours,total

hours,rate->total

phone->eid,name (?????)

Boyce-Codd Normal Form (BCNF)

For a relation R, with FDs of the form $X \rightarrow Y$, it is in BCNF iff

Every FD is either:

- 1) Trivial (e.g., Y contains X) ($SSN \rightarrow SSN, Name$)
- 2) X is a key of the table

- If an FD violates (2), multiple rows with same X value may occur
 - Indicates redundancy, as rows with given X value all have same Y value
 - E.g., $SSN \rightarrow Name, Addr$ in non-decomposed hobbies schema
 - SSN is not a key of the whole table
 - Name, Addr repeated for each appearance of a given SSN
- To put a schema into BCNF, create subtables of form XY
 - E.g., tables where key is left side (X) of one or more FDs
 - Repeat until all tables in BCNF
 - Effectively eliminates redundancy, while preserving (most) dependencies

BCNFify

BCNFify(schema **R**, functional dependency set **F**):

D = {(**R**,**F**)} // **D** is set of output relations

while there is a (schema ,FD set) pair (**S**,**F'**) in **D** not in BCNF, do:

 given **X**→**Y** as a BCNF-violating dependency in **F'**

 replace (**S**,**F'**) in **D** with

S1 = (**XY**,**F1**) and

S2 = ((**S**-**Y**) ∪ **X**, **F2**)

 where **F1** and **F2** are the FDs in **F'** over **XY** or (**S**-**Y**) ∪ **X**,

 respectively

End

return **D**

BCNFify Example for Hobbies

Iter 1

S = SSN, H = Hobby, N = Name, A = Addr, C = Cost

Schema	FDs
(<u>S</u> , H)	$S, H \rightarrow N, A, C$
	$S \rightarrow N, A$
	$H \rightarrow C$

key

violates bcnf

Iter 2

Schema	FDs
(<u>S</u> , N, A)	$S \rightarrow N, A$

Schema	FDs
(S, <u>H</u>)	$S, H \rightarrow C$
	$H \rightarrow C$

violates bcnf

$S, H \rightarrow N, A, C$ implies
 $S, H \rightarrow C$

Did we lose $S, H \rightarrow N, A, C$? No!

$S, H \rightarrow N, A, C$ is implied by $H \rightarrow C$
 and $S \rightarrow N, A$

No, using union rule from
 Armstrong's Axioms,

$S+H \rightarrow N, A + C$

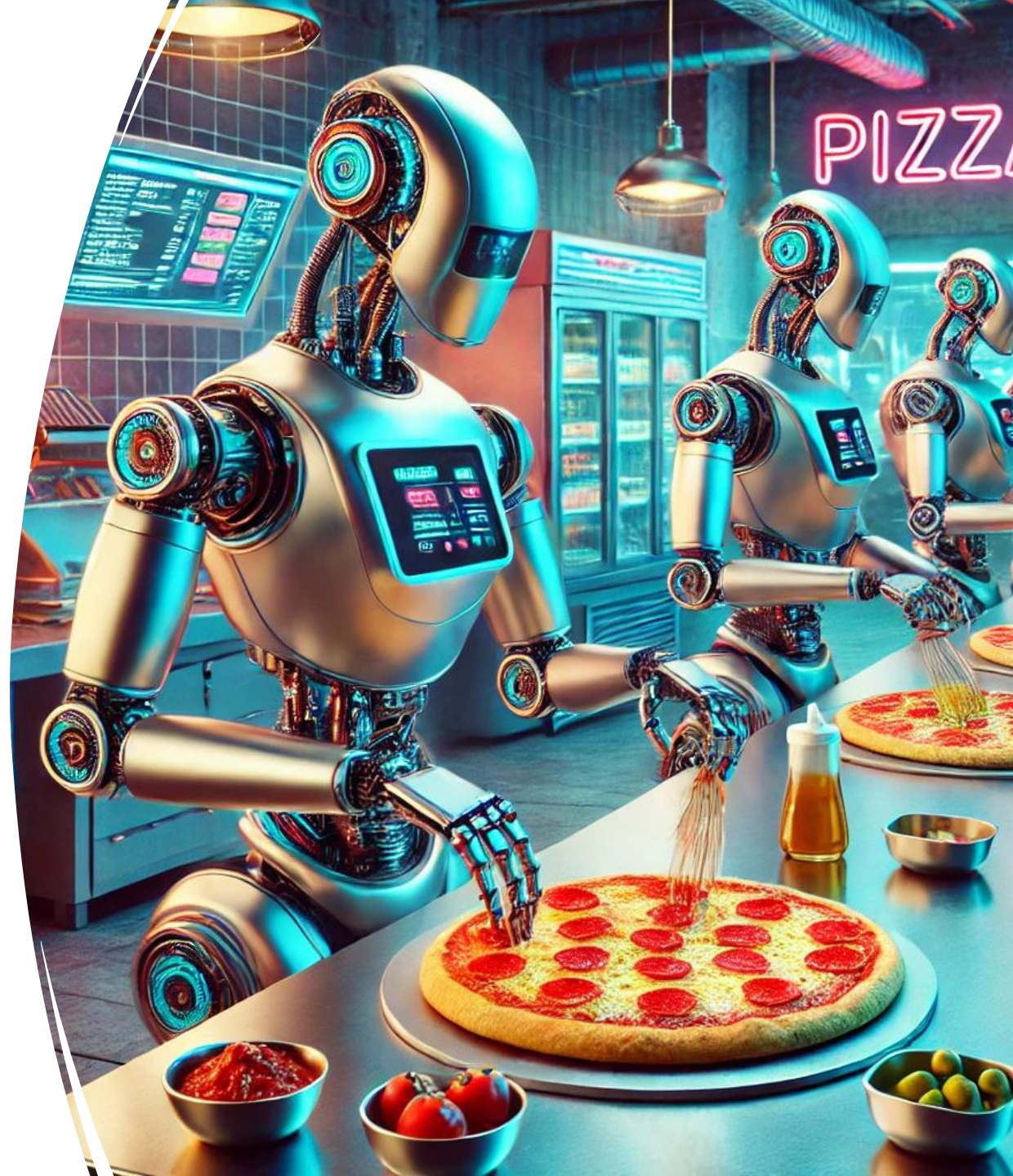
3 remaining tables are same as
 in ER decomposition

Schema	FDs
(<u>H</u> , C)	$H \rightarrow C$

Schema	FDs
(<u>S</u> , H)	

BCNF vs 3NF

A new pizzeria startup called Cheese, Veggies, and Meat specializes on making only Pizzas that allow to have 3 toppings: at most 1 cheese, at most 1 veggie, and at most 1 meat.



BCNF vs 3NF

A new robot pizzeria startup called Cheese, Veggies, and Meat uses 3 robots: one to add cheese, another to add meat, and a third to add veggies. Unfortunately, their current prototype can add at most one type of cheese, one type of veggies, and one type of meat per pizza.

<u>id</u>	topping	<u>type</u>
1	Mozzarella	Cheese
1	Bacon	Meat
1	Spinach	Veggie
2	Mozzarella	Cheese
2	Sausage	Meat
2	Spinach	Veggie

```
CREATE TABLE pizza{
  id INT,
  topping varchar[30],
  type varchar[30],
  PRIMARY KEY (id, type)
}
```


BCNF vs 3NF

A new robot pizzeria startup called Cheese, Veggies, and Meat uses 3 robots: one to add cheese, another to add meat, and a third to add veggies. Unfortunately, their current prototype can add at most one type of cheese, one type of veggies, and one type of meat per pizza.

<u>id</u>	topping	<u>type</u>
1	Mozzarella	Cheese
1	Bacon	Meat
1	Spinach	Veggie
2	Mozzarella	Cheese
2	Sausage	Meat
2	Spinach	Veggie

Redundancy!

```
CREATE TABLE pizza{
  id INT,
  topping varchar[30],
  type varchar[30],
  PRIMARY KEY (id, type)
}
```

This schema is in 3NF but not BCNF

BCNF vs 3NF

A new robot pizzeria startup called Cheese, Veggies, and Meat uses 3 robots: one to add cheese, another to add meat, and a third to add veggies. Unfortunately, their current prototype can add at most one type of cheese, one type of veggies, and one type of meat per pizza.

<u>id</u>	topping	<u>type</u>
1	Mozzarella	Cheese
1	Bacon	Meat
1	Spinach	Veggie
2	Mozzarella	Cheese
2	Sausage	Meat
2	Spinach	Veggie

Redundancy!

```
CREATE TABLE pizza{
  id INT,
  topping varchar[30],
  type varchar[30],
  PRIMARY KEY (id, type)
}
```

FD:

ID, Type -> {}

Type -> topping

This schema is in 3NF but not BCNF

BCNF vs 3NF

A new robot pizzeria startup called Cheese, Veggies, and Meat uses 3 robots: one to add cheese, another to add meat, and a third to add veggies. Unfortunately, their current prototype can add at most one type of cheese, one type of veggies, and one type of meat per pizza.

<u>id</u>	<u>topping</u>
1	Mozzarella
1	Bacon
1	Spinach
2	Mozzarella
2	Sausage
2	Spinach
2	Sausage

<u>topping</u>	<u>tuype</u>
Mozzarella	Cheese
Bacon	Meat
Spinach	Veggie
Sausage	Meat

Primary key on the table no longer prevents adding this row (dependencies are lost)

No redundancy and in BCNF but how do we enforce now that we have at most 1 of each topping type?

BCNF vs 3NF

- BCNF will get rid of anomalies but is not “dependency preserving”
- 3rd Normal Form (3NF) eliminates as much redundancy as possible while preserving all dependencies
 - We will skip the details
- Neither form is “better”
 - You can choose **either** dependency preservation (3NF) or redundancy-free (BCNF)

Study Break

- Patient database
- Want to represent patients at hospitals with doctors
- Patients have names, birthdates
- Doctors have names, specialties
- Hospitals have names, addresses
- One doctor can treat multiple patients, each patient has one doctor
- Each patient in one hospital, hospitals have many patients

1) Draw an ER diagram

2) What are the functional dependencies

3) What is the normalized schema? Is it redundancy free?

Solution

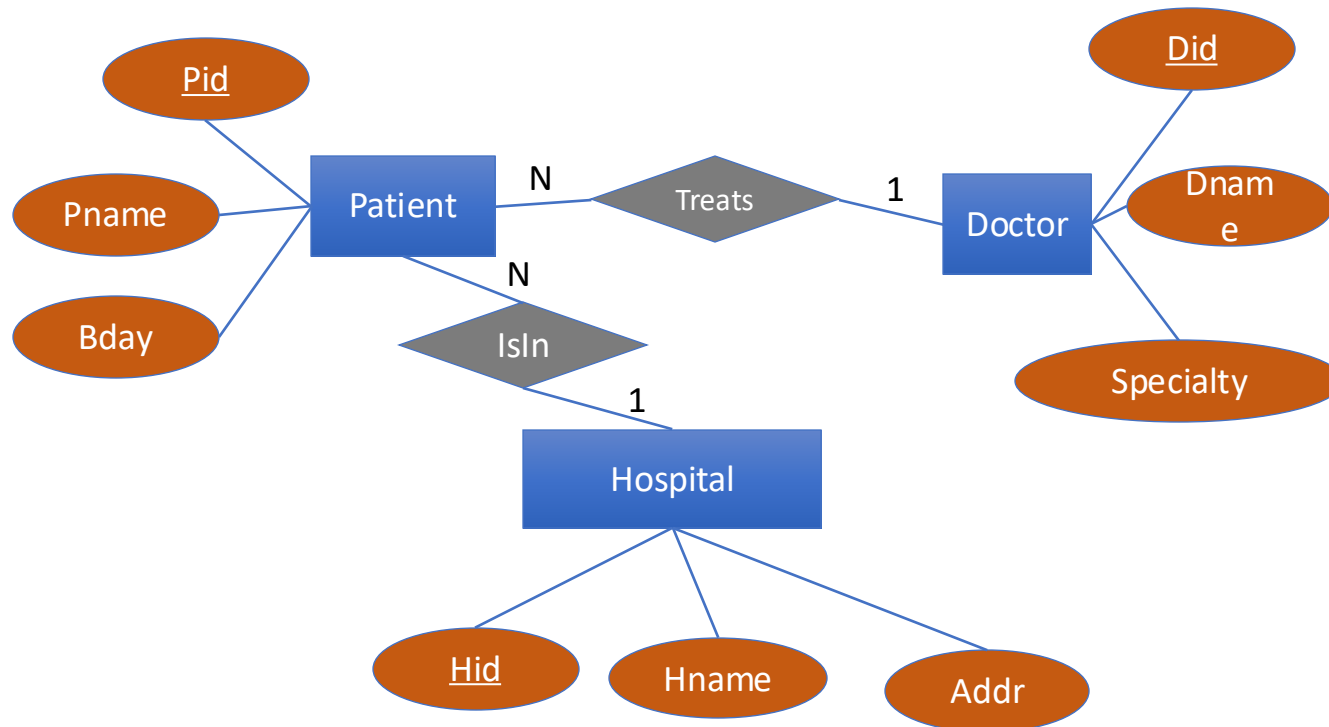
Patients have names, birthdates

Doctors have names, specialties

Hospitals have names, addresses

One doctor can treat multiple patients, each patient has one doctor

Each patient in one hospital, hospitals have many patients



Pid → Pname, Bday, P_Did, P_Hid

Hid → Hname, Addr

Did → Dname, Specialty

Patients (Pid, Pname, Bday, P_Did, P_Hid)

Hospitals (Hid, Hname, Addr)

Doctors (Did, Dname, Specialty)

Recap

- Properly normalized schemas avoid redundancy and preserve dependencies
- Functional dependencies and normal forms (e.g., BCNF) give us a formal way to reason about these concepts
- In practice, people use ER modeling to derive a schema in BCNF rather than the BCNFify algorithm